

CURSO DE GRADUAÇÃO EM BACHARELADO DE CONTROLE E AUTOMAÇÃO

LUIZ HENRIQUE GARIGLIO DOS SANTOS

PROJETO CONTROLE DE ESTOQUE

MINAS GERAIS
2022

LUIZ HENRIQUE GARIGLIO DOS SANTOS

PROJETO CONTROLE DE ESTOQUE

Trabalho apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de Minas Gerais, para a disciplina de Algoritmos 2.

Professor: Luiz Guilherme Hilel Drumond Silveira.

Sumário

1	INTRODUÇÃO	3
2	DESCRIÇÃO DO PROJETO	4
3	ESTRUTURA DO PROJETO	5
3.1	BIBLIOTECAS	5
3.2	ESTRUTURAS	5
3.2.1	Produto	5
3.2.2	Célula	6
3.2.3	Estoque	6
3.2.4	Produto Venda	6
3.2.5	Célula Venda	7
3.2.6	Pilha Venda	7
3.3	FUNÇÕES e Procedimentos	8
3.3.1	Iniciar Funções	8
3.3.2	Função Main	8
3.3.3	Procedimento Inicio	9
3.3.4	Procedimento Iniciar Estoque	10
3.3.5	Procedimento Iniciar Venda	10
3.3.6	Procedimento Operações Produtos	11
3.3.7	Procedimento Registro Produtos	12
3.3.8	Procedimento Alterar Produtos	13
3.3.9	Procedimento Alterar Estoque	14
3.3.10	Procedimento Consultar Estoque	15
3.3.11	Procedimento Mostrar Estoque	16
3.3.12	Procedimento Buscar Produto	17
3.3.13	Função Realiza Venda	18
3.3.14	Função Insere Produtos Venda	19
3.3.15	Procedimento Mostrar Venda	20
3.3.16	Função Remove Produto Venda	20
3.3.17	Procedimento Remove Venda Estoque	21
3.3.18	Procedimento Fazer Backup	22

1 INTRODUÇÃO

O controle de estoque é uma parte importante dos mais variados negócios. Tem o objetivo de garantir o sucesso das operações e diminuir erros. Permite, por exemplo, evitar compras em excesso, a ausência de um determinado produto e erros na determinação dos preços. O projeto, então, consiste na construção de um algoritmo que realiza controle de estoque. Será desenvolvido na linguagem de programação C++ por meio do Visual Studio.

Será entregue a disciplina de Algoritmos 2, ministrada pelo professor Luiz Guilherme Hilel Drumond Silveira, do curso de Bacharelado em Engenharia de Controle e Automação no Instituto Federal de Minas Gerais, Campus Sabará.

Desse modo, o presente trabalho tem como objetivo, apresentar a ideia, desenvolvimento e resultado do projeto.

2 DESCRIÇÃO DO PROJETO

Para que o controle de estoque ocorra de forma satisfatória, sem erros, e com as devidas funcionalidades, o algoritmo deve conter algumas funções básicas, dentre elas:

- Registro de produtos: É necessário que produtos que antes não faziam parte do catálogo possam ser incluídos. Assim, deve permitir ao usuário inserir novos itens e suas informações básicas, como nome, quantidade, custo, preço e código.
- Movimentações de estoque: As movimentações são importantes pois permitem que o estoque seja ajustado e represente a quantidade real de produtos disponíveis. Podem ocorrer de forma a permitir a adição, por compras, e retirada, por vendas ou descarte por perda.
- Estoque: Aqui, é mostrada a quantidade de produtos disponíveis a venda. É resultado do valor indicado inicialmente na quantidade, subtraídas as retiradas e somadas os acréscimos.

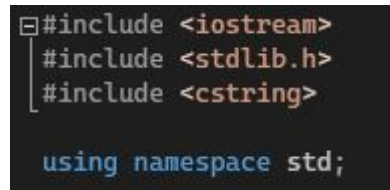
As ações que o programa proporciona serão construídas no algoritmo por meio de funções e procedimentos. As operações ocorrerão de forma manual através das opções selecionadas pelo usuário. E as informações dos produtos serão salvas por meio de "*structs*" que contém as informações básicas.

3 ESTRUTURA DO PROJETO

3.1 BIBLIOTECAS

As bibliotecas utilizadas para o desenvolvimento do código, Figura 1, foram incluídas no início do código sendo elas `"iostream"`, `"stdlib.h"` e `"cstring"`.

Figura 1 – Foto Bibliotecas no código



```
#include <iostream>
#include <stdlib.h>
#include <cstring>

using namespace std;
```

Fonte: Autoria Própria, 2022

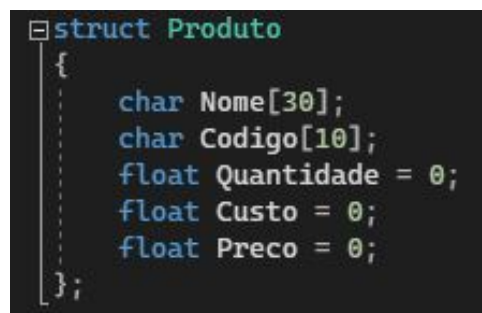
3.2 ESTRUTURAS

Para o desenvolvimento do algoritmo, foi necessária a determinação de algumas *structs*, que armazenam diferentes tipos de dados. As estruturas criadas e seus objetivos serão descritos a seguir.

3.2.1 Produto

A primeira *struct* desenvolvida foi a denominada Produto, figura 2, criada para satisfazer a necessidade do projeto de se registrar os produtos com diversas informações, por se tratar de controle de estoque. Por isso, nela estão contidas o nome do produto, o código, a quantidade disponível no estoque, o custo e o preço de venda do produto.

Figura 2 – Foto struct produto



```
struct Produto
{
    char Nome[30];
    char Codigo[10];
    float Quantidade = 0;
    float Custo = 0;
    float Preco = 0;
};
```

Fonte: Autoria Própria, 2022

3.2.2 Célula

A *struct* Celula (figura 3) tem o objetivo de servir como célula da estrutura de fila, meio pelo qual será organizado o estoque. Por isso, ela armazena uma variável do tipo Produto, cada célula armazena um produto, e um ponteiro do próprio tipo Celula chamado próximo, que durante a implementação do código, aponta para a próxima célula.

Figura 3 – Foto struct celula

```
struct celula
{
    struct celula* proximo;
    struct Produto produto;
};
```

Fonte: Autoria Própria, 2022

3.2.3 Estoque

A *struct* Estoque (figura 4), é a última necessária para a construção da estrutura de fila. É ela quem aponta para todas as células que contém o estoque. Por isso, contém três variáveis, sendo duas do tipo ponteiro de Celula, para que um aponte para a primeira posição, início, e outro para o último, fim. A terceira é do tipo inteiro e contém a quantidade de itens que compõem o estoque.

Figura 4 – Foto struct estoque

```
struct Fila_Estoque
{
    struct celula* inicio;
    struct celula* fim;
    int quantidade;
};
```

Fonte: Autoria Própria, 2022

3.2.4 Produto Venda

Para que fosse possível implementar vendas no algoritmo, foi precisa a utilização de outra estrutura, sendo a escolhida de pilha. Para isso, foi determinada primeiro, uma nova estrutura, produto_venda (figura 5), apenas com as informações necessárias a venda, nome, código, quantidade disponível, preço e total.

Figura 5 – Foto struct produto_venda

```
struct produto_venda
{
    char Nome[30];
    char Codigo[10];
    float Quantidade = 0;
    float Preco = 0;
    float total = 0;
};
```

Fonte: Autoria Própria, 2022

3.2.5 Célula Venda

O próximo passo para a criação da pilha para vendas, foi definir uma nova *struct* para as células, denominada *celula_venda* (figura 6). Ela, do mesmo modo da célula do estoque, possui um item do tipo *produto_venda*, e outro do tipo ponteiro de *celula_venda*, que aponta para as outras células.

Figura 6 – Foto struct celula_venda

```
struct celula_venda {
    struct celula_venda* proximo;
    struct produto_venda item;
};
```

Fonte: Autoria Própria, 2022

3.2.6 Pilha Venda

Por fim, a *struct* *pilha_venda* realiza o controle da pilha e contém dois itens, um ponteiro do tipo *celula_venda*, que aponta para a célula do topo, e outro item do tipo inteiro que contém a quantidade de células na pilha.

Figura 7 – Foto struct pilha_venda

```
struct Pilha_Venda {
    struct celula_venda* topo;
    int quantidade;
};
```

Fonte: Autoria Própria, 2022

3.3 FUNÇÕES e Procedimentos

3.3.1 Iniciar Funções

A imagem abaixo mostra o trecho de código em que todas as funções utilizadas no programa são inicializadas.

Figura 8 – Foto Iniciar funções

```
/*Iniciando Funções*/  
  
void Inicio();  
  
void IniciarEstoque(struct Fila_Estoque* e);  
  
void Operacoes_Produtos(struct Fila_Estoque* e);  
void Registro_Produtos(struct Fila_Estoque* e);  
void Alterar_Produto(struct Fila_Estoque* e);  
  
void Consultar_Estoque(struct Fila_Estoque* e);  
void Mostrar_Estoque(struct Fila_Estoque* e);  
void Buscar_Produto(struct Fila_Estoque* e);  
void Alterar_Estoque(struct Fila_Estoque* e);  
  
void IniciarVenda(struct Pilha_Venda* v);  
  
float Realizar_Venda(struct Pilha_Venda* v, struct Fila_Estoque* e);  
float Insere_Produto_Venda(struct Pilha_Venda* v, struct Fila_Estoque* e, float total_venda);  
float Remove_Produto_Venda(struct Pilha_Venda* v, float total_venda);  
void Mostrar_Venda(struct Pilha_Venda* v, float total_venda);  
void Remove_Venda_Estoque(struct Pilha_Venda* v, struct Fila_Estoque* e);  
  
void Fazer_Backup(struct Fila_Estoque* e, struct Fila_Estoque* b);
```

Fonte: Autoria Própria, 2022

3.3.2 Função Main

A função main (figura 9) é a função principal do código. Nela, ocorre a alocação das estruturas Estoque, Backup e Venda, o usuário determina a ação que deseja fazer e realiza a chamada das outras funções. O processo para escolha da ação ocorre em *loop* e só termina quando o usuário decide sair.

Figura 9 – Foto Função Main

```
int main()
{
    int acao = 0, sair = 0;
    float Saldo = 0;
    struct Fila_Estoque* Estoque;
    Estoque = (Fila_Estoque*)malloc(sizeof(Fila_Estoque));
    IniciarEstoque(Estoque);

    struct Fila_Estoque* Backup;
    Backup = (Fila_Estoque*)malloc(sizeof(Fila_Estoque));

    struct Pilha_Venda* Venda;
    Venda = (Pilha_Venda*)malloc(sizeof(Pilha_Venda));
    IniciarVenda(Venda);

    while (acao != 8)
    {
        Inicio();
        cin >> acao;
        if (acao == 1)
        {
            Operacoes_Produtos(Estoque);
        }
        else if (acao == 2)
        {
            Alterar_Estoque(Estoque);
        }
        else if (acao == 3)
        {
            Consultar_Estoque(Estoque);
        }
        else if (acao == 4)
        {
            Saldo = Saldo + Realizar_Venda(Venda, Estoque);
            Remove_Venda_Estoque(Venda, Estoque);
            IniciarVenda(Venda);
        }
        else if (acao == 5)
        {
            Fazer_Backup(Estoque, Backup);
        }
        else if (acao == 6)
        {
            Mostrar_Estoque(Backup);
        }
        else if (acao == 7)
        {
            cout << "\n\t0 Saldo e de: " << Saldo << " reais.";
            cout << "\n\tPressione 1 para sair.\n\t";
            cin >> sair;
            cout << "\n\tSaindo...";
        }
        else if (acao == 8)
        {
            cout << "\n\tSaindo ...";
        }
        else {
            cout << "\n\tDigite uma entrada valida!";
        }
    }

    return 0;
}
```

Fonte: Autoria Própria, 2022

3.3.3 Procedimento Inicio

O procedimento Inicio (figura 10) É sempre chamado no início do *loop* da função main pois é quem imprime a página inicial com as opções que o usuário pode escolher.

Figura 10 – Foto Procedimento Inicio

```
void Inicio()
{
    system("clear || cls");
    cout << "\n\tPrograma Controle de Estoque - Luiz Henrique Gariglio dos Santos\n";
    cout << "\n\tDigite o que deseja fazer:\n";
    cout << "\n\t1 - Produto;";
    cout << "\n\t2 - Movimentacoes de Estoque;";
    cout << "\n\t3 - Consultar estoque;";
    cout << "\n\t4 - Realizar venda;";
    cout << "\n\t5 - Realizar backup;";
    cout << "\n\t6 - Mostrar backup;";
    cout << "\n\t7 - Mostrar saldo;";
    cout << "\n\t8 - Sair.\n\n\t";
}
```

Fonte: Autoria Própria, 2022

3.3.4 Procedimento Iniciar Estoque

O procedimento `IniciarEstoque`, figura 11, é o responsável por inicializar a estrutura do tipo `fila`, `estoque`. Nele, os ponteiros `inicio` e `fim` apontam para `"NULL"` e a quantidade é zerada. É pedido como parâmetro um ponteiro `struct Fila_Estoque`.

Figura 11 – Foto Procedimento iniciar estoque

```
void IniciarEstoque(struct Fila_Estoque* e)
{
    e->inicio = NULL;
    e->fim = NULL;
    e->quantidade = 0;
}
```

Fonte: Autoria Própria, 2022

3.3.5 Procedimento Iniciar Venda

O procedimento `IniciarVenda`, figura 12, do mesmo modo de `IniciarEstoque`, inicializa uma estrutura. Contudo, é a estrutura do tipo `pilha`, `venda`. O ponteiro `topo` aponta para `"NULL"` e a quantidade é zerada. O parâmetro necessário é um ponteiro `struct Pilha_Venda`.

Figura 12 – Foto Procedimento iniciar venda

```
void IniciarVenda(struct Pilha_Venda* v)
{
    v->topo = NULL;
    v->quantidade = 0;
}
```

Fonte: Autoria Própria, 2022

3.3.6 Procedimento Operações Produtos

Ao seleccionar no menu a opção Produto, é chamado o procedimento "Operacoes_Produtos"(imagem 13) e passado como parâmetro o ponteiro struct Fila_Estoque. Ele abre um novo menu com as possíveis ações relacionadas a produtos e chama as devidas funções.

Figura 13 – Foto Procedimento de operações de produtos

```
void Operacoes_Produtos(struct Fila_Estoque* e)
{
    int acao_P = 0;

    while (acao_P != 3)
    {
        system("clear || cls");
        cout << "\n\tProdutos\n";
        cout << "\n\tDigite o que deseja fazer:\n";
        cout << "\n\t1 - Registrar Produto;";
        cout << "\n\t2 - Alterar Produto;";
        cout << "\n\t3 - Sair.\n\n\t";
        cin >> acao_P;

        if (acao_P == 1)
        {
            Registro_Produtos(e);
        }
        else if (acao_P == 2)
        {
            Alterar_Produto(e);
        }
        else if (acao_P == 3)
        {
            cout << "\n\tSaindo ...";
        }
    }
}
```

Fonte: Autoria Própria, 2022

3.3.7 Procedimento Registro Produtos

No menu "Produto", a primeira opção a ser selecionada é a de registro de produtos. Assim, é chamado o procedimento "Registro_Produtos"(imagem 14) e passado como parâmetro o ponteiro *struct* Fila_Estoque. Nesse procedimento, o usuário passa todas as informações necessárias para registrar o produto, que é registrado em uma célula e adicionado a fila.

Figura 14 – Foto Procedimento de registro de produtos

```
void Registro_Produtos(struct Fila_Estoque* e)
{
    struct celula* novo;
    struct Produto novop;
    novo = (struct celula*)malloc(sizeof(struct celula));

    system("clear || cls");
    cout << "\n\tDigite o nome do produto: ";
    cin >> novop.Nome;
    cout << "\n\tDigite o código do produto: ";
    cin >> novop.Codigo;
    cout << "\n\tDigite a quantidade: ";
    cin >> novop.Quantidade;
    cout << "\n\tDigite o custo do produto: ";
    cin >> novop.Custo;
    cout << "\n\tDigite o preço do produto: ";
    cin >> novop.Preco;

    novo->proximo = NULL;
    novo->produto = novop;

    if (e->quantidade > 0)
    {
        e->fim->proximo = novo;
        e->fim = novo;
        e->quantidade++;
    }
    else
    {
        e->inicio = novo;
        e->fim = novo;
        e->quantidade++;
    }
}
```

Fonte: Autoria Própria, 2022

3.3.8 Procedimento Alterar Produtos

Outra opção que pode ser selecionada no menu "Produto" é a de alterar produtos. É chamado o procedimento "Alterar_Produtos", figura 15, e passado também a fila estoque. O usuário passa o nome ou código do produto que deseja alterar, o algoritmo realiza a busca e, se encontra, permite ao usuário alterar as informações de nome, código, custo e preço do produto.

Figura 15 – Foto Procedimento de alteração de produtos

```
void Alterar_Produto(struct Fila_Estoque* e)
{
    struct celula* aux;
    aux = e->inicio;
    char nome[30];
    int sair_AP = 0;
    system("clear || cls");

    if (e->quantidade == 0)
    {
        cout << "\n\tNao ha produtos registrados!";
    }
    else
    {
        cout << "\n\tDigite o nome ou o codigo do produto que deseja alterar: ";
        cin >> nome;

        while (aux != NULL && strcmp(aux->produto.Nome, nome) != 0 && strcmp(aux->produto.Codigo, nome))
        {
            aux = aux->proximo;
        }

        if (aux == NULL)
        {
            cout << "\n\tProduto nao encontrado!";
            cout << "\n\tPressione 1 para sair.\n\t";
            cin >> sair_AP;
            if (sair_AP == 1)
            {
                cout << "\n\tSaindo...";
            }
        }
    }

    else
    {
        //mostra produto antigo
        cout << "\n\t Nome: " << aux->produto.Nome;
        cout << "\n\t Codigo: " << aux->produto.Codigo;
        cout << "\n\t Quantidade no estoque: " << aux->produto.Quantidade;
        cout << "\n\t Custo do produto: " << aux->produto.Custo;
        cout << "\n\t Preco de venda: " << aux->produto.Preco;
        cout << endl;

        //insere e realiza as alteracoes
        cout << "\n\n\tDigite as alteracoes";
        cout << "\n\tDigite o nome do produto: ";
        cin >> aux->produto.Nome;
        cout << "\n\tDigite o codigo do produto: ";
        cin >> aux->produto.Codigo;
        cout << "\n\tDigite o custo do produto: ";
        cin >> aux->produto.Custo;
        cout << "\n\tDigite o preco do produto: ";
        cin >> aux->produto.Preco;

        cout << "\n\tAlteracao realizada com sucesso!";
        cout << "\n\tPressione 1 para sair.\n\t";
        cin >> sair_AP;
        if (sair_AP == 1) { ... }
    }
}
```

Fonte: Autoria Própria, 2022

3.3.9 Procedimento Alterar Estoque

Caso selecionada a opção no menu principal de movimentações de estoque. É chamado o procedimento `Alterar_Estoque`, figura 16, e passado o ponteiro da fila estoque. Nele, o usuário informa nome ou código do produto que deseja alterar. Depois, é questionado se a ação é de entrada ou saída, e questionada a quantidade. Caso seja selecionada a opção de entrada, o algoritmo soma o valor indicada com a quantidade já existente no produto. Caso seja selecionada a retirada, é subtraído o valor indicado da quantidade presente no estoque.

Figura 16 – Foto Procedimento de movimentação de estoque

```
void Alterar_Estoque(struct Fila_Estoque* e)
{
    int movimentacao = 0, quantidade = 0;

    system("clear || cls");
    char nome[30];
    int sair_E = 0;
    struct celula* aux;
    aux = e->inicio;

    if (e->quantidade == 0)
    {
        cout << "\n\tNao ha produtos registrados!";
    }
    else
    {
        cout << "\n\tDigite o nome ou o codigo do produto que deseja alterar: ";
        cin >> nome;

        while (aux != NULL && strcmp(aux->produto.Nome, nome) != 0 && strcmp(aux->produto.Codigo, nome) != 0)
        {
            aux = aux->proximo;
        }
        if (aux == NULL)
        {
            cout << "\n\tProduto não encontrado!";
        }
    }

    else
    {
        while (movimentacao != 1 && movimentacao != 2)
        {
            cout << "\n\tSelecione:\n\t1-Entrada\n\t2-Retirada\n\t";
            cin >> movimentacao;

            if (movimentacao == 1)
            {
                cout << "\n\tRealizar entrada em " << aux->produto.Nome;
                cout << "\n\tDigite a quantidade: ";
                cin >> quantidade;
                aux->produto.Quantidade = aux->produto.Quantidade + quantidade;
                cout << "\n\tAlteracao realizada com sucesso!";
            }
            else if (movimentacao == 2)
            {
                cout << "\n\tRealizar retirada em " << aux->produto.Nome;
                cout << "\n\tDigite a quantidade: ";
                cin >> quantidade;
                aux->produto.Quantidade = aux->produto.Quantidade - quantidade;
                cout << "\n\tAlteracao realizada com sucesso!";
            }
            else
            {
                cout << "\n\tDigite uma entrada valida!";
            }
        }
    }
}
```

Fonte: Autoria Própria, 2022

3.3.10 Procedimento Consultar Estoque

No menu principal, é possível selecionar a opção de consultar estoque. É então chamado o procedimento "Consultar_Estoque", imagem 17, e passado a pilha estoque. Ele abre um novo menu com as opções relacionadas ao estoque.

Figura 17 – Foto Procedimento Consultar Estoque

```
void Consultar_Estoque(struct Fila_Estoque* e)
{
    int acao_E = 0;

    while (acao_E != 3)
    {
        system("clear || cls");
        cout << "\n\tEstoque\n";
        cout << "\n\tDigite o que deseja fazer:\n";
        cout << "\n\t1 - Mostrar estoque;";
        cout << "\n\t2 - Buscar Produto;";
        cout << "\n\t3 - Sair.\n\n\t";
        cin >> acao_E;

        if (acao_E == 1)
        {
            Mostrar_Estoque(e);
        }
        else if (acao_E == 2)
        {
            Buscar_Produto(e);
        }
        else if (acao_E == 3)
        {
            cout << "\n\tSaindo ...";
        }
    }
}
```

Fonte: Autoria Própria, 2022

3.3.11 Procedimento Mostrar Estoque

Caso seja selecionada a opção mostrar estoque, é chamado o procedimento `Mostrar_Estoque`, figura 18, e passada a fila estoque. O procedimento passa por toda a fila e a imprime na tela.

Figura 18 – Foto Procedimento Mostrar Estoque

```
void Consultar_Estoque(struct Fila_Estoque* e)
{
    int acao_E = 0;

    while (acao_E != 3)
    {
        system("clear || cls");
        cout << "\n\tEstoque\n";
        cout << "\n\tDigite o que deseja fazer:\n";
        cout << "\n\t1 - Mostrar estoque;\n";
        cout << "\n\t2 - Buscar Produto;\n";
        cout << "\n\t3 - Sair.\n\n\t";
        cin >> acao_E;

        if (acao_E == 1)
        {
            Mostrar_Estoque(e);
        }
        else if (acao_E == 2)
        {
            Buscar_Produto(e);
        }
        else if (acao_E == 3)
        {
            cout << "\n\tSaindo ...";
        }
    }
}
```

Fonte: Autoria Própria, 2022

3.3.12 Procedimento Buscar Produto

Ao selecionar essa opção, é chamado o procedimento `Buscar_Produto`, imagem 19, que pede ao usuário que informe o nome ou código do produto que deseja buscar. Então, percorre toda a fila até encontrar o produto desejado e imprime suas informações.

Figura 19 – Foto Procedimento Buscar Produto

```
void Buscar_Produto(struct Fila_Estoque* e)
{
    system("clear || cls");
    char nome[30];
    int sair_E = 0;
    struct celula* aux;
    aux = e->inicio;

    if (e->quantidade == 0)
    {
        cout << "\n\tNao ha produtos registrados!";
    }
    else
    {
        cout << "\n\tDigite o nome ou o codigo do produto que deseja buscar: ";
        cin >> nome;

        while (aux != NULL && strcmp(aux->produto.Nome, nome) != 0 && strcmp(aux->produto.Codigo, nome) != 0)
        {
            aux = aux->proximo;
        }
        if (aux == NULL)
        {
            cout << "\n\tProduto não encontrado!";
        }
        else
        {
            cout << "\n\t Nome: " << aux->produto.Nome;
            cout << "\n\t Codigo: " << aux->produto.Codigo;
            cout << "\n\t Quantidade no estoque: " << aux->produto.Quantidade;
            cout << "\n\t Custo do produto: " << aux->produto.Custo;
            cout << "\n\t Preco de venda: " << aux->produto.Preco;
            cout << endl;

            cout << "\n\tPressione 1 para sair.\n\t";
            cin >> sair_E;
            if (sair_E == 1)
            {
                cout << "\n\tSaindo...";
            }
        }
    }
}
```

Fonte: Autoria Própria, 2022

3.3.13 Função Realiza Venda

Essa opção do menu principal chama a função `Realizar_Venda`, imagem 20, que têm como parâmetros os ponteiros fila estoque e pilha venda e retorna um *float* relativo ao valor total da venda. Abre um novo menu com as opções para realização da venda.

Figura 20 – Foto Função Realizar Venda

```
float Realizar_Venda(struct Pilha_Venda* v, struct Fila_Estoque* e)
{
    system("clear || cls");
    cout << "\n\tNova venda";

    int acao = 0;
    float total_venda = 0;

    while (acao != 2)
    {
        if (v->quantidade != 0)
        {
            Mostrar_Venda(v, total_venda);

            cout << "\n\t1-Novo Produto\n\t2-Finalizar venda\n\t3-Excluir ultimo produto\n\t";
            cin >> acao;

            if (acao == 1)
            {
                total_venda = Insere_Produto_Venda(v, e, total_venda);
            }
            else if (acao == 2)
            {
                cout << "\n\tSaindo ...";
            }
            else if (acao == 3)
            {
                total_venda = Remove_Produto_Venda(v, total_venda);
            }
            else {
                cout << "\n\tDigite uma entrada valida!";
            }
        }

        return total_venda;
    }
}
```

Fonte: Autoria Própria, 2022

3.3.14 Função Insere Produtos Venda

Já no menu de venda, ao ser selecionada essa opção, é chamada a função `Insere_Produto_Venda`, figura 21. Têm como parâmetros as estruturas de pilha venda e fila estoque e o float `total_venda`. O usuário informa o produto e a quantidade, o algoritmo busca o produto na fila de estoque e, se encontra, copia algumas das informações para a `celula_venda`, como nome, código e preço. E o total é calculado multiplicando a quantidade pelo preço. A variável `total_venda` é somada ao total do produto e retornada pela função.

Figura 21 – Foto Função Insere Produto Venda

```
float Insere_Produto_Venda(struct Pilha_Venda* v, struct Fila_Estoque* e, float total_venda)
{
    struct celula_venda* novo = NULL;
    novo = (struct celula_venda*)malloc(sizeof(struct celula_venda));

    struct celula* aux;
    aux = e->inicio;

    char nome[30];

    cout << "\n\tDigite o nome ou código do produto:";
    cin >> nome;

    while (aux != NULL && strcmp(aux->produto.Nome, nome) != 0 && strcmp(aux->produto.Codigo, nome) != 0)
    {
        aux = aux->proximo;
    }

    if (aux == NULL)
    {
        cout << "\n\tProduto não encontrado!";
    }

    else
    {
        strcpy_s(novo->item.Nome, aux->produto.Nome);
        strcpy_s(novo->item.Codigo, aux->produto.Codigo);
        cout << "\n\tDigite a Quantidade: ";
        cin >> novo->item.Quantidade;
        novo->item.Preco = aux->produto.Preco;
        novo->item.total = novo->item.Quantidade * novo->item.Preco;
        novo->proximo = NULL;
        total_venda = total_venda + novo->item.total;

        v->quantidade++;

        if (v->quantidade == 0)
        {
            novo->proximo = NULL;
            v->topo = novo;
        }
        else
        {
            novo->proximo = v->topo;
            v->topo = novo;
        }
    }

    return total_venda;
}
```

Fonte: Autoria Própria, 2022

3.3.15 Procedimento Mostrar Venda

Esse procedimento não pode ser selecionado pelo usuário. Ela é sempre chamada, caso já existam produtos na pilha de venda, enquanto a venda não é finalizada. Ela é a responsável por imprimir na tela os produtos vendidos bem como a quantidade, o preço unitário, o total de cada produto e o total da venda.

Figura 22 – Foto Procedimento Mostrar Venda

```
void Mostrar_Venda(struct Pilha_Venda* v, float total_venda)
{
    system("clear || cls");
    cout << "\n\n\t***** Produtos *****";

    // ...

    struct celula_venda* aux;
    aux = v->topo;
    int i = 0;

    while (i < v->quantidade)
    {
        i++;
        cout << "\n\t" << i << " - Nome: " << aux->item.Nome;
        cout << "\t\tQuantidade: " << aux->item.Quantidade;
        cout << "\t\tPreco: " << aux->item.Preco;
        cout << "\t\tValor: " << aux->item.total;

        aux = aux->proximo;
    }

    cout << "\n\n\t\t Total venda: " << total_venda << endl;
}
```

Fonte: Autoria Própria, 2022

3.3.16 Função Remove Produto Venda

Ao selecionar a opção excluir último produto, é chamada a função Remove_Produto_Venda. Nela, o último item adicionado, topo da pilha, é removido e o total do produto é removido do valor total da venda. Tem como parâmetros a pilha de vendas e o valor total da venda. E retorna o valor total da venda já atualizado.

Figura 23 – Foto Função Remove Produto Venda

```
float Remove_Produto_Venda(struct Pilha_Venda* v, float total_venda)
{
    struct celula_venda* aux;
    aux = v->topo;
    cout << aux->item.total;
    total_venda = total_venda - aux->item.total;
    v->topo = v->topo->proximo;
    free(aux);
    v->quantidade--;
    return total_venda;
}
```

Fonte: Autoria Própria, 2022

3.3.17 Procedimento Remove Venda Estoque

Após a função de realizar venda ser finalizada, é chamado o procedimento Retirar_Venda_Estoque. Os parâmetros passados são a pilha venda e a fila estoque. Esse procedimento é o responsável por subtrair os itens vendidos pelo estoque. Para isso, percorre toda a pilha de venda e a cada um de seus itens, o busca no estoque para realizar a subtração das quantidades.

Figura 24 – Foto Função Remove Venda Estoque

```
void Remove_Venda_Estoque(struct Pilha_Venda* v, struct Fila_Estoque* e)
{
    int i = 0;
    cout << v->quantidade;
    if (v->quantidade != 0)
    {
        struct celula* aux;
        aux = (struct celula*)malloc(sizeof(struct celula));

        struct celula_venda* aux2;
        aux2 = (struct celula_venda*)malloc(sizeof(struct celula_venda));
        aux2 = v->topo;
        int i = 0;
        while (i < v->quantidade)
        {
            aux = e->inicio;
            while (strcmp(aux->produto.Nome, aux2->item.Nome) != 0)
            {
                aux = aux->proximo;
            }
            aux->produto.Quantidade = aux->produto.Quantidade - aux2->item.Quantidade;
            aux2 = aux2->proximo;
            i++;
        }
    }
}
```

Fonte: Autoria Própria, 2022

3.3.18 Procedimento Fazer Backup

O procedimento Fazer_Backup, imagem 25, realiza uma cópia da fila estoque. Para isso, recebe como parâmetro a fila estoque e outra fila backup, do mesmo tipo de estoque.

Figura 25 – Foto Procedimento Fazer Backup

```
void Fazer_Backup(struct Fila_Estoque* e, struct Fila_Estoque* b )
{
    int i = 0;
    struct Produto p;
    struct celula* aux;
    aux = (struct celula*)malloc(sizeof(struct celula));
    aux = e->inicio;

    IniciarEstoque(b);

    while (i < e->quantidade)
    {
        struct celula* aux2;
        aux2 = (struct celula*)malloc(sizeof(struct celula));

        strcpy_s(p.Nome, aux->produto.Nome);
        strcpy_s(p.Codigo, aux->produto.Codigo);
        p.Custo = aux->produto.Custo;
        p.Preco = aux->produto.Preco;
        p.Quantidade = aux->produto.Quantidade;

        aux2->produto = p;
        aux2->proximo = NULL;

        if (b->quantidade > 0)
        {
            b->fim->proximo = aux2;
            b->fim = aux2;
            b->quantidade++;
        }

        else
        {
            b->inicio = aux2;
            b->fim = aux2;
            b->quantidade++;
        }

        aux = aux->proximo;
        i++;
    }
}
```

Fonte: Autoria Própria, 2022